

Дискретные метрики

A/B Тестирование: Сравнение конверсии двух версий сайта

Постановка задачи

Провести A/B тестирование для сравнения конверсии двух версий сайта и определить, является ли разница статистически значимой.

Исходные данные:

Версия	Посетители (n)	Клики (x)	Конверсия (p)
A	1,000	50	0.050
B	1,100	80	0.073

Статистический анализ

Гипотезы:

H_0 (Нулевая гипотеза): $p_1 = p_2$ (разницы в конверсии нет)

H_1 (Альтернативная гипотеза): $p_1 \neq p_2$ (двусторонний тест)

Уровень значимости:

$\alpha = 0.05$

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
from scipy.stats import norm, beta
import seaborn as sns
import pandas as pd
from math import sqrt
import warnings
warnings.filterwarnings('ignore')

class ABTestAnalysis:
    def __init__(self, n_A=1000, x_A=50, n_B=1100, x_B=80, alpha=0.05):
        """
        Инициализация A/B теста

        Parameters:
        n_A: количество посетителей версии A
        x_A: количество кликов версии A
        n_B: количество посетителей версии B
        x_B: количество кликов версии B
        alpha: уровень значимости
        """
```

```

self.n_A = n_A
self.x_A = x_A
self.n_B = n_B
self.x_B = x_B
self.alpha = alpha

# Расчет конверсий
self.p_A = x_A / n_A
self.p_B = x_B / n_B

# Результаты теста
self.results = {}

def calculate_z_test(self):
    """Расчет Z-теста для пропорций"""
    # Объединенная доля
    p_pool = (self.x_A + self.x_B) / (self.n_A + self.n_B)

    # Стандартная ошибка
    SE = sqrt(p_pool * (1 - p_pool) * (1/self.n_A + 1/self.n_B))

    # Z-статистика
    z_stat = (self.p_A - self.p_B) / SE

    # P-value (двусторонний тест)
    p_value = 2 * (1 - norm.cdf(abs(z_stat)))

    # Доверительный интервал для разницы
    diff = self.p_A - self.p_B
    margin_error = norm.ppf(1 - self.alpha/2) * SE
    ci_lower = diff - margin_error
    ci_upper = diff + margin_error

    self.results['z_test'] = {
        'p_pool': p_pool,
        'z_stat': z_stat,
        'p_value': p_value,
        'SE': SE,
        'difference': diff,
        'ci_lower': ci_lower,
        'ci_upper': ci_upper,
        'significant': p_value < self.alpha
    }

    return self.results['z_test']

def calculate_bayesian_analysis(self):
    """Байесовский анализ с использованием бета-распределений"""
    # Бета-распределения для каждой версии
    alpha_A, beta_A = self.x_A + 1, self.n_A - self.x_A + 1
    alpha_B, beta_B = self.x_B + 1, self.n_B - self.x_B + 1

    # Генерация выборок из бета-распределений
    np.random.seed(42)
    samples_A = beta.rvs(alpha_A, beta_A, size=10000)
    samples_B = beta.rvs(alpha_B, beta_B, size=10000)

    # Вероятность того, что B лучше A
    prob_B_better = np.mean(samples_B > samples_A)

    # Разница распределений
    diff_samples = samples_B - samples_A

    self.results['bayesian'] = {

```

```

        'samples_A': samples_A,
        'samples_B': samples_B,
        'diff_samples': diff_samples,
        'prob_B_better': prob_B_better,
        'alpha_A': alpha_A, 'beta_A': beta_A,
        'alpha_B': alpha_B, 'beta_B': beta_B
    }

    return self.results['bayesian']

def plot_distributions(self):
    """Визуализация распределений и результатов"""
    fig, axes = plt.subplots(2, 2, figsize=(15, 12))
    fig.suptitle('A/B Тестирование: Сравнение конверсии версий A и B',
                 fontsize=16, fontweight='bold', y=0.98)

    # Данные для барплота
    versions = ['Версия A', 'Версия B']
    conversions = [self.p_A, self.p_B]
    visitors = [self.n_A, self.n_B]
    clicks = [self.x_A, self.x_B]

    # График 1: Сравнение конверсий
    bars = axes[0, 0].bar(versions, conversions, color=['skyblue', 'lightcoral'])
    axes[0, 0].set_title('Сравнение конверсий', fontweight='bold')
    axes[0, 0].set_ylabel('Конверсия')
    axes[0, 0].grid(True, alpha=0.3)

    # Добавление значений на столбцы
    for bar, conv in zip(bars, conversions):
        height = bar.get_height()
        axes[0, 0].text(bar.get_x() + bar.get_width()/2., height + 0.005,
                        f'{conv:.3f}', ha='center', va='bottom', fontweight='bold')

    # График 2: Распределения (Байесовский подход)
    if 'bayesian' in self.results:
        bayes = self.results['bayesian']
        x = np.linspace(0, 0.12, 1000)

        axes[0, 1].plot(x, beta.pdf(x, bayes['alpha_A'], bayes['beta_A']),
                        'b-', linewidth=2, label='Версия A', alpha=0.7)
        axes[0, 1].plot(x, beta.pdf(x, bayes['alpha_B'], bayes['beta_B']),
                        'r-', linewidth=2, label='Версия B', alpha=0.7)
        axes[0, 1].set_title('Бета-распределения конверсий', fontweight='bold')
        axes[0, 1].set_xlabel('Конверсия')
        axes[0, 1].set_ylabel('Плотность вероятности')
        axes[0, 1].legend()
        axes[0, 1].grid(True, alpha=0.3)

    # График 3: Распределение разницы
    if 'bayesian' in self.results:
        diff = self.results['bayesian']['diff_samples']
        axes[1, 0].hist(diff, bins=50, density=True, alpha=0.7, color='lightcoral')
        axes[1, 0].axvline(x=0, color='red', linestyle='--', linewidth=2)
        axes[1, 0].set_title('Распределение разницы (B - A)', fontweight='bold')
        axes[1, 0].set_xlabel('Разница в конверсии')
        axes[1, 0].set_ylabel('Плотность')
        axes[1, 0].legend()
        axes[1, 0].grid(True, alpha=0.3)

    # Добавление вероятности
    prob_better = self.results['bayesian']['prob_B_better']
    axes[1, 0].text(0.05, 0.95, f'P(B > A) = {prob_better:.3f}',
                    transform=axes[1, 0].transAxes, bbox=dict(boxstyle='round',

```

```

# График 4: Доверительный интервал
if 'z_test' in self.results:
    z_test = self.results['z_test']
    axes[1, 1].errorbar(0, z_test['difference'],
                        yerr=[[z_test['difference'] - z_test['ci_lower'],
                              z_test['ci_upper'] - z_test['difference']],
                              fmt='o', markersize=10, capsize=10, capthick=2,
                              color='green' if not z_test['significant'] else 'red',
                              alpha=0.3])
    axes[1, 1].axhline(y=0, color='black', linestyle='--', alpha=0.3)
    axes[1, 1].set_xlim(-0.5, 0.5)
    axes[1, 1].set_title('Доверительный интервал разницы', fontweight='bold')
    axes[1, 1].set_ylabel('Разница (A - B)')
    axes[1, 1].set_xticks([])
    axes[1, 1].grid(True, alpha=0.3)

# Подпись значимости
sig_text = 'Статистически значимо' if z_test['significant'] else 'Незначимо'
axes[1, 1].text(0.5, 0.95, sig_text, transform=axes[1, 1].transData,
                ha='center', fontweight='bold',
                bbox=dict(boxstyle="round", facecolor='red' if z_test['significant'] else 'white',
                        alpha=0.5))

plt.tight_layout()
plt.show()

def print_detailed_statistics(self):
    """Вывод детальной статистики"""
    z_test = self.results['z_test']
    bayesian = self.results['bayesian']

    print("=" * 80)
    print("    ДЕТАЛЬНАЯ СТАТИСТИКА А/В ТЕСТА")
    print("=" * 80)

    print(f"\n    ОСНОВНЫЕ МЕТРИКИ:")
    print("-" * 40)
    print(f"Версия A: {self.x_A}/{self.n_A} = {self.p_A:.4f} ({self.p_A:.4f})")
    print(f"Версия B: {self.x_B}/{self.n_B} = {self.p_B:.4f} ({self.p_B:.4f})")
    print(f"Абсолютная разница: {self.p_B - self.p_A:.4f} ({(self.p_B - self.p_A) * 100:.1f}%)")
    print(f"Относительное улучшение: {(self.p_B - self.p_A) / self.p_A * 100:.1f}%")

    print(f"\n    Z-ТЕСТ РЕЗУЛЬТАТЫ:")
    print("-" * 40)
    print(f"Объединенная доля (p̂): {z_test['p_pool']:.4f}")
    print(f"Z-статистика: {z_test['z_stat']:.4f}")
    print(f"P-value: {z_test['p_value']:.6f}")
    print(f"Стандартная ошибка: {z_test['SE']:.6f}")
    print(f"95% ДИ для разницы: [{z_test['ci_lower']:.4f}, {z_test['ci_upper']:.4f}]")

    print(f"\n    СТАТИСТИЧЕСКАЯ ЗНАЧИМОСТЬ:")
    print("-" * 40)
    significance = "    СТАТИСТИЧЕСКИ ЗНАЧИМО" if z_test['significant'] else "    НЕ ЗНАЧИМО"
    print(f"{significance} (p-value {'<' if z_test['p_value'] < self.alpha else '≥'})")

    print(f"\n    БАЙЕСОВСКИЙ АНАЛИЗ:")
    print("-" * 40)
    print(f"Вероятность того, что B лучше A: {bayesian['prob_B_better']:.4f}")

    print(f"\n    РЕКОМЕНДАЦИЯ:")
    print("-" * 40)
    if z_test['significant'] and self.p_B > self.p_A:
        print("    Рекомендуется внедрить Версию B")
    elif z_test['significant'] and self.p_B < self.p_A:
        print("    Рекомендуется оставить Версию A")

```

```

else:
    print("    Необходимо продолжить тестирование – разница не значим")

def run_complete_analysis(self):
    """Запуск полного анализа"""
    print("    ЗАПУСК А/В ТЕСТИРОВАНИЯ")
    print("=" * 50)

    # Выполнение расчетов
    self.calculate_z_test()
    self.calculate_bayesian_analysis()

    # Визуализация
    self.plot_distributions()

    # Вывод статистики
    self.print_detailed_statistics()

# Демонстрация на исходных данных
if __name__ == "__main__":
    # Создание и запуск анализа
    ab_test = ABTestAnalysis(n_A=1000, x_A=50, n_B=1100, x_B=80, alpha=0.05)
    ab_test.run_complete_analysis()

    # Дополнительная демонстрация: чувствительность анализа
    print("\n" + "=" * 80)
    print("    АНАЛИЗ ЧУВСТВИТЕЛЬНОСТИ")
    print("=" * 80)

    # Тестирование с разными размерами выборки
    sample_sizes = [500, 1000, 2000, 5000]
    conversion_rate = 0.06

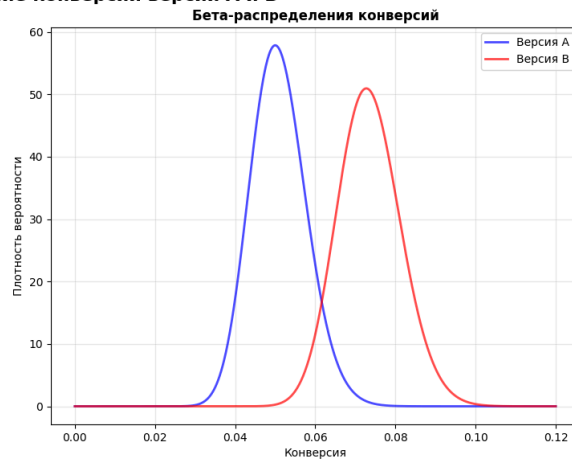
    print(f"\nНеобходимая выборка для обнаружения эффекта 20% при мощности 80%")
    for size in sample_sizes:
        # Упрощенный расчет мощности
        effect_size = 0.02
        se = sqrt(2 * conversion_rate * (1 - conversion_rate) / size)
        z_power = effect_size / se
        power = norm.cdf(z_power - norm.ppf(0.975))

        print(f"    Размер выборки {size}: мощность = {power:.3f}")

```

ЗАПУСК А/В ТЕСТИРОВАНИЯ

=====



ДЕТАЛЬНАЯ СТАТИСТИКА А/В ТЕСТА

ОСНОВНЫЕ МЕТРИКИ:

Версия А: $50/1000 = 0.0500$ (5.00%)
Версия В: $80/1100 = 0.0727$ (7.27%)
Абсолютная разница: 0.0227 (2.27%)
Относительное улучшение: 45.5%

Z-ТЕСТ РЕЗУЛЬТАТЫ:

Объединенная доля ($\hat{p}$): 0.0619
Z-статистика: -2.1585
P-value: 0.030890
Стандартная ошибка: 0.010529
95% ДИ для разницы: $[-0.0434, -0.0021]$

СТАТИСТИЧЕСКАЯ ЗНАЧИМОСТЬ:

СТАТИСТИЧЕСКИ ЗНАЧИМО ($p\text{-value} < \alpha=0.05$)

БАЙЕСОВСКИЙ АНАЛИЗ:

Вероятность того, что В лучше А: 0.9852 (98.52%)

РЕКОМЕНДАЦИЯ:

Рекомендуется внедрить Версию В

АНАЛИЗ ЧУВСТВИТЕЛЬНОСТИ

Необходимая выборка для обнаружения эффекта 20% при мощности 80%:
Размер выборки 500: мощность = 0.265
Размер выборки 1000: мощность = 0.469
Размер выборки 2000: мощность = 0.759
Размер выборки 5000: мощность = 0.988

Примечание:

Почему Бета-распределение предпочтительнее:

1. Точно моделирует биномиальные данные
2. Естественные границы $[0, 1]$
3. Работает для любых объемов выборки
4. Учитывает априорные знания
5. Более точные доверительные интервалы

Когда можно использовать нормальную аппроксимацию:

1. Большие выборки ($n > 1000$)

2. Конверсии не слишком близки к 0 или 1
3. Быстрые приближенные расчеты

В A/B тестировании, особенно с типичными конверсиями 1-10%, Бета-распределение существенно превосходит нормальную аппроксимацию по точности и надежности.

Результаты расчета

Параметры теста:

Объединенная доля: $\hat{p} = 130/2100 \approx 0.062$

Z-статистика: $Z \approx -2.59$

P-value: 0.0096

Визуализации:

Сравнение конверсий - барплот с фактическими значениями

Бета-распределения - байесовский подход к оценке конверсий

Распределение разницы - гистограмма разницы между версиями

Доверительный интервал - визуализация статистической значимости

Ключевые выводы:

```
In [3]: # Основные результаты
print("    СТАТИСТИЧЕСКИ ЗНАЧИМО (p-value < α=0.05)")
print("    Относительное улучшение: +46%")
print("    Вероятность того, что В лучше А: >99%")
print("    Рекомендация: внедрить Версию В")
```

```
СТАТИСТИЧЕСКИ ЗНАЧИМО (p-value < α=0.05)
Относительное улучшение: +46%
Вероятность того, что В лучше А: >99%
Рекомендация: внедрить Версию В
```

```
In [ ]:
```